uses a transaction to perform these operations, so if any of them fail, all the others are rolled back, thereby leaving the database unmodified.

The service is almost ready to replace the `DbContext` operations in the `PageModel`. Before you can use the service in code, you must register it with the service container. Go to Program.cs, and add the following registration:

```
builder.Services.AddScoped<IPropertyService, PropertyService>();
```

8.5.1 Modifying data

Turning to the scaffolded `EditModel` class, you will make the following alteration to replace the existing private field with new ones for an `IPropertyService` and an `ICityService`. The injected context is replaced with the services. You also need to add a `using` directive to reference `CityBreaks.Services`.

Listing 8.26 Injecting the `IPropertyService` in place of the `DbContext`

```
private readonly IPropertyService _propertyService;
private readonly ICityService _cityService;

public EditModel(IPropertyService propertyService, ICityService cityService)
{
    _propertyService = propertyService;
    _cityService = cityService;
}
```

In the step shown in listing 8.27, you replace the `Property` binding target with individual binding targets representing the values you want to expose to the user. You also add a public `SelectList` property to replace the `ViewData` approach currently being taken for the city dropdown. Finally, you replace the `OnGet` handler parameter (`id`) with a